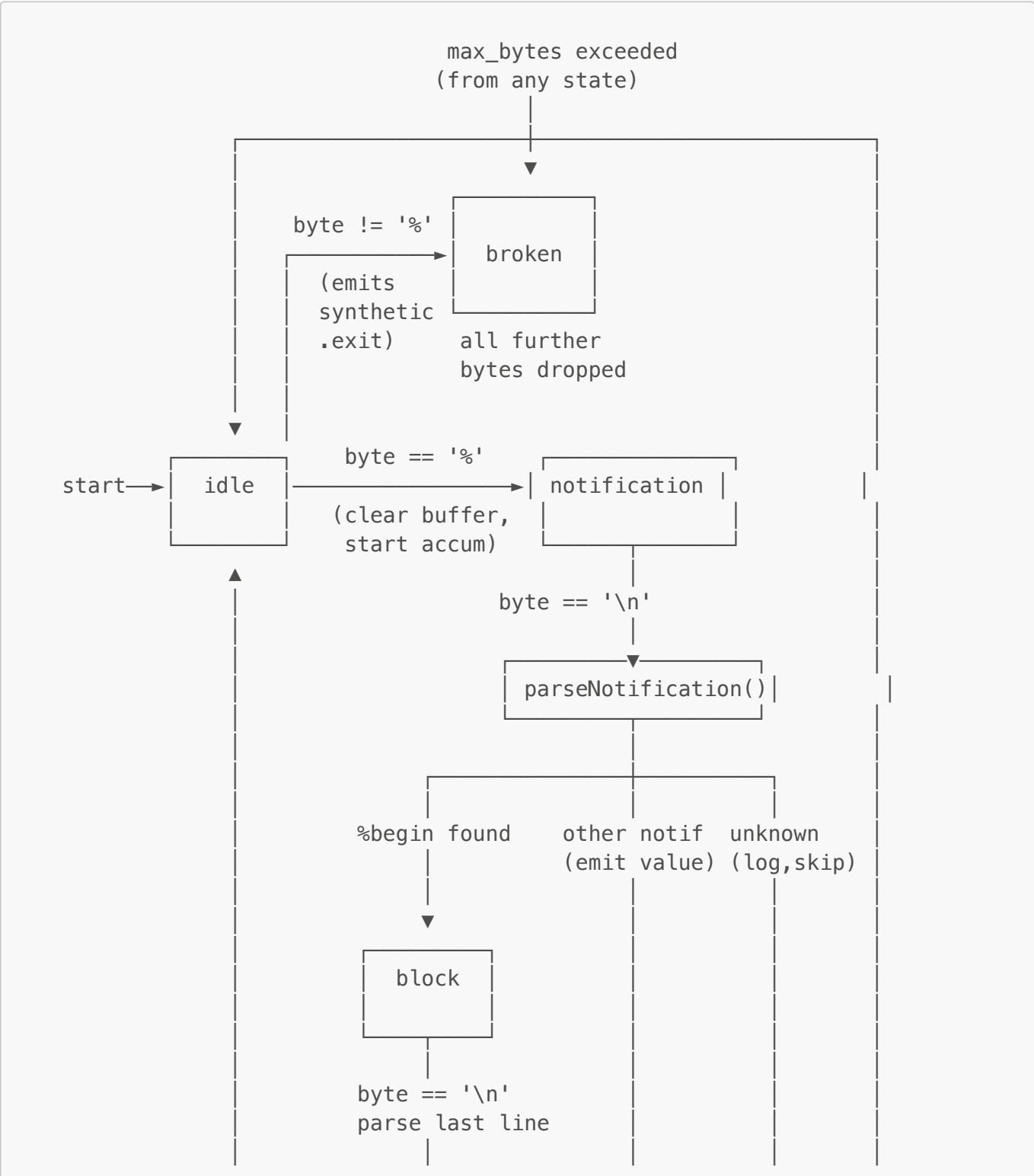


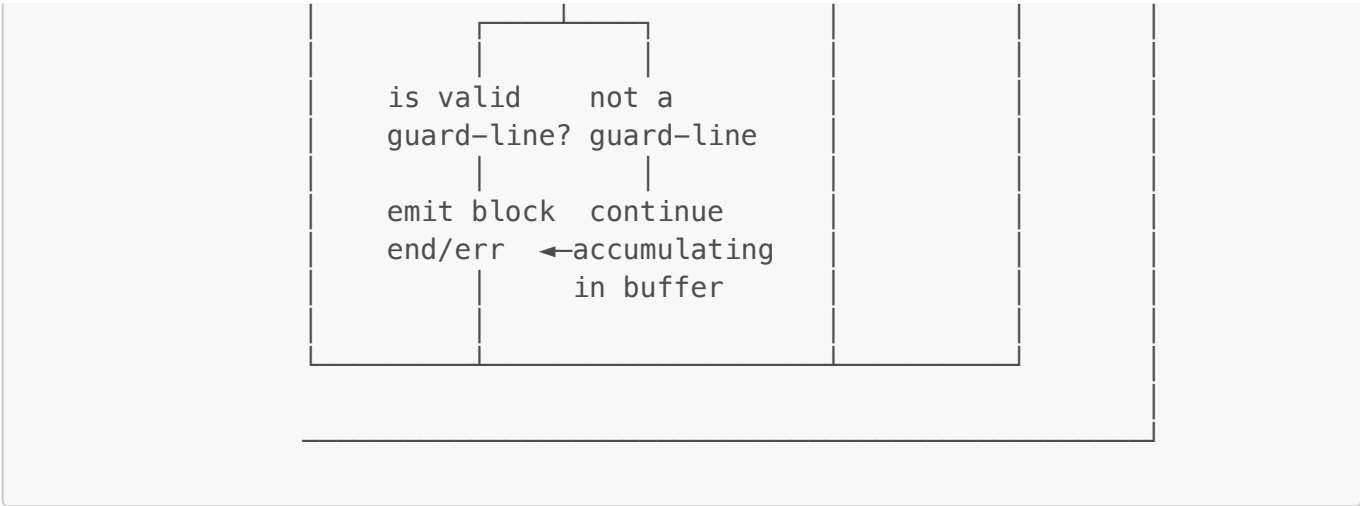
Ghostty tmux Control Mode: Technical Diagrams

All diagrams referenced in the deep analysis (Sections 6-14). These are meant to be precise technical references for implementation work.

1. Parser State Machine (`control.zig`)

The control mode protocol parser has 4 states. Every byte from the tmux control mode stream passes through this state machine.





State Details

State	Entry condition	Behavior	Exit conditions
idle	Initial state; after emitting any notification	Waits for % byte	% byte → notification; non-% byte → broken (emit synthetic exit)
notification	% byte received in idle	Accumulates bytes into buffer	\n → parse accumulated line, transition based on result
block	%begin parsed in notification	Accumulates block payload lines	\n + valid guard-line (%end/%error with matching format) → emit block_end/block_err → idle
broken	Non-% byte in idle, or max_bytes exceeded	Terminal state — drops all input	None (unrecoverable)

Guard-Line Validation (Critical for Correctness)

A line inside a %begin/%end block is only a valid guard-line if ALL of:

- 1. Starts with %end or %error
- 2. Has exactly 4 whitespace-separated tokens: %end <timestamp> <command_number> <flags>
- 3. <timestamp> is a valid integer
- 4. <command_number> is a valid integer
- 5. <flags> is a valid integer

This prevents false termination when shell output inside a block coincidentally starts with %end.

Example of a trap:

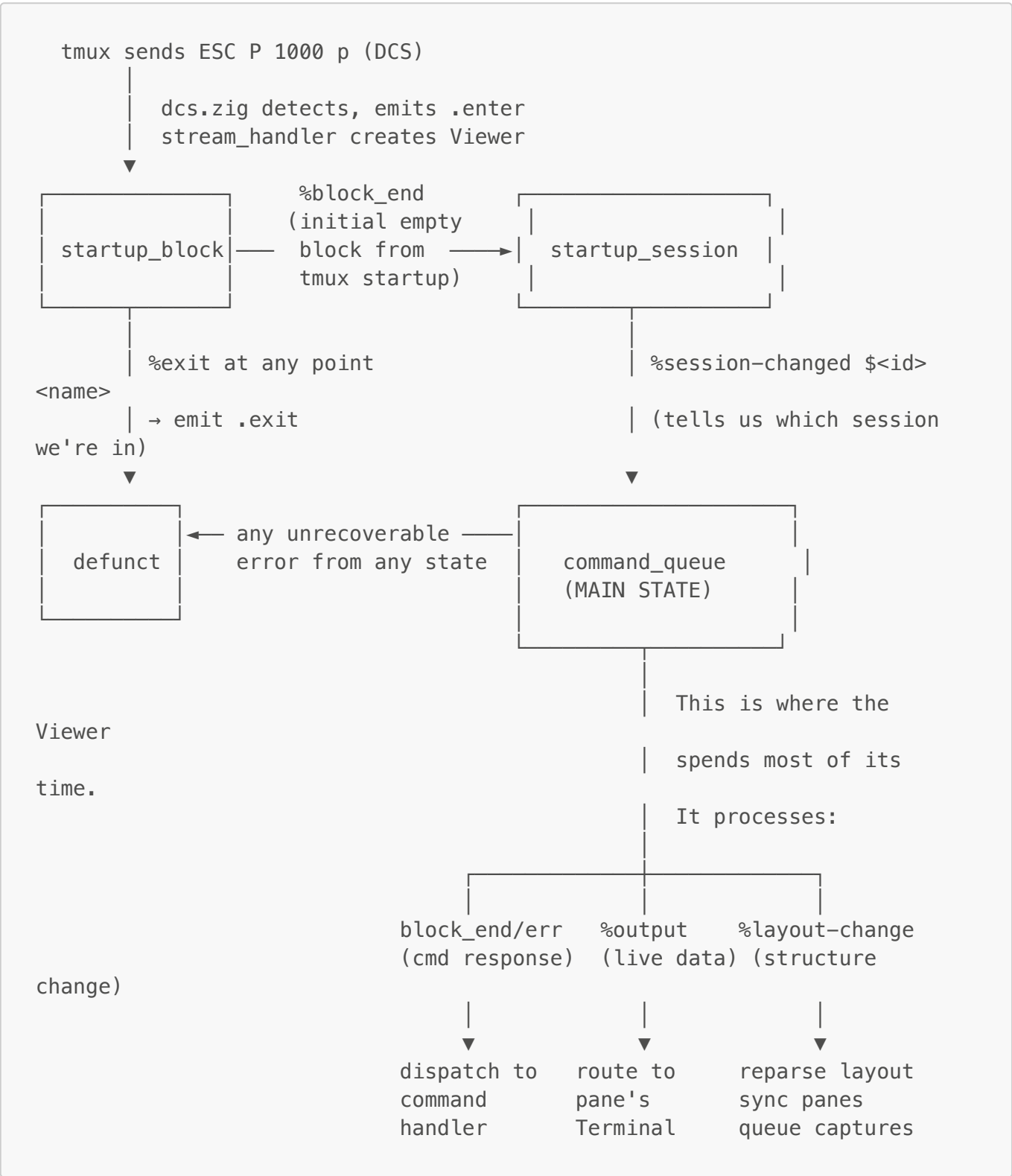
```
%begin 1000 5 0
The line below is NOT a guard-line:
%end of file reached
```

```
This IS the real guard-line:
%end 1000 5 0
```

2. Viewer State Machine (viewer.zig)

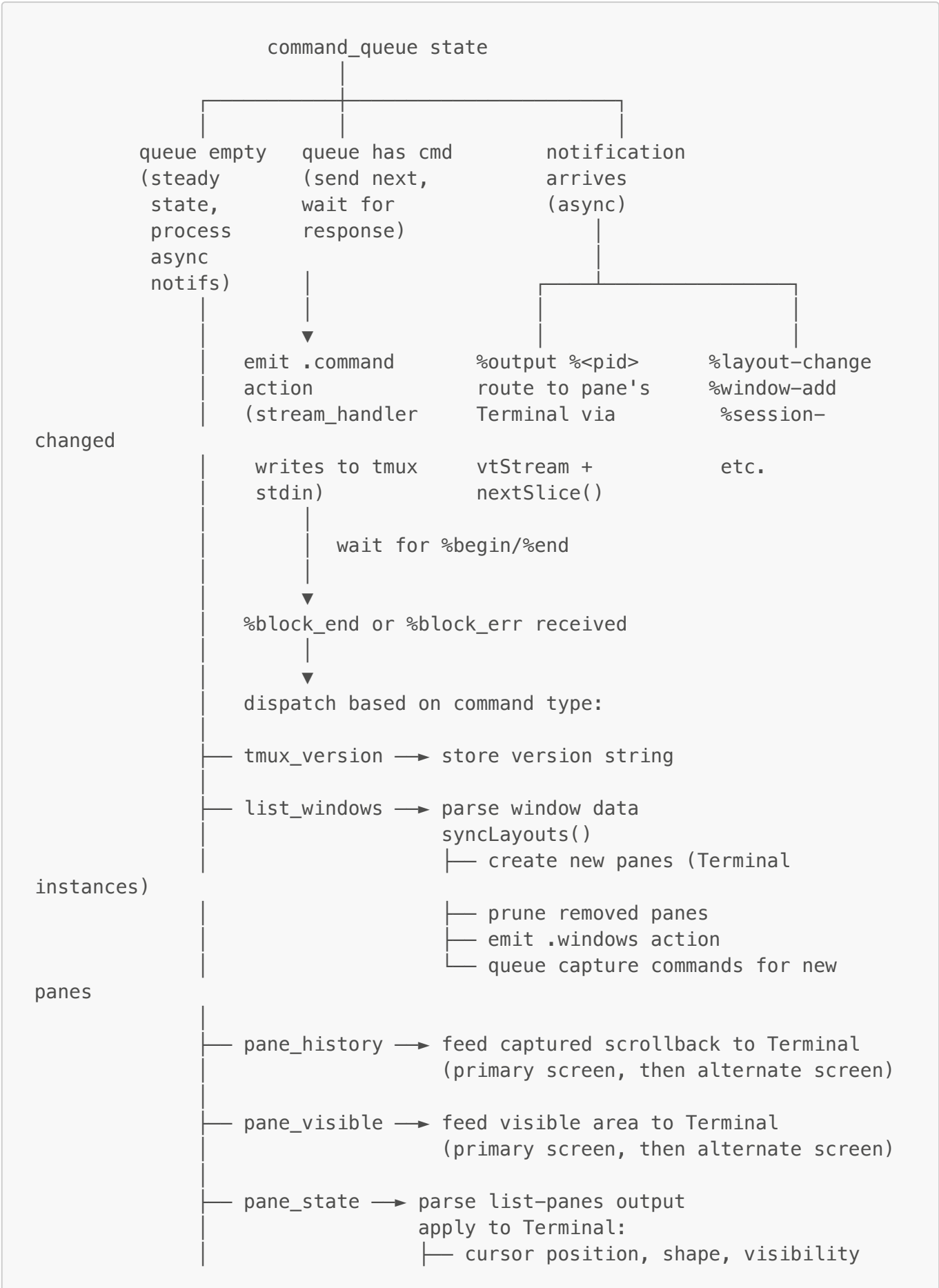
The Viewer is the "brain" that orchestrates a tmux control mode session from DCS detection through steady-state operation.

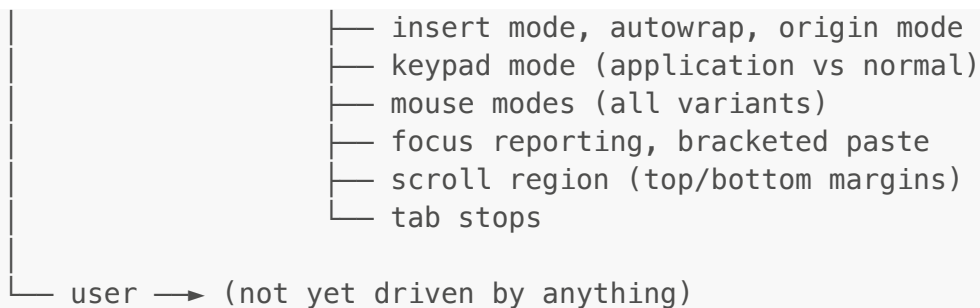
2.1 High-Level Lifecycle



2.2 Command Queue Detail

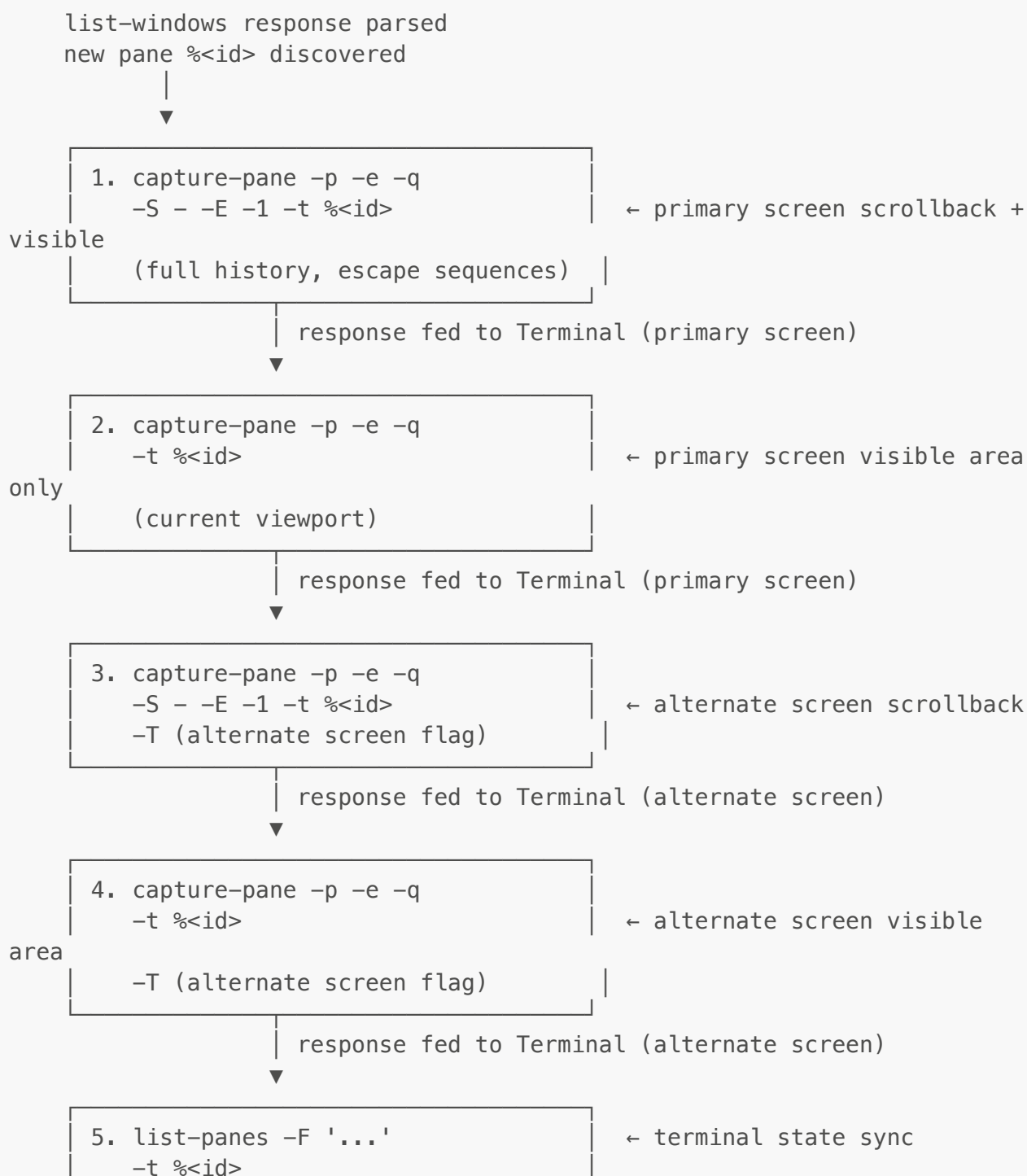
The Viewer sends one command at a time to tmux and waits for the %begin/%end response before sending the next. This is the serialized command processing flow.

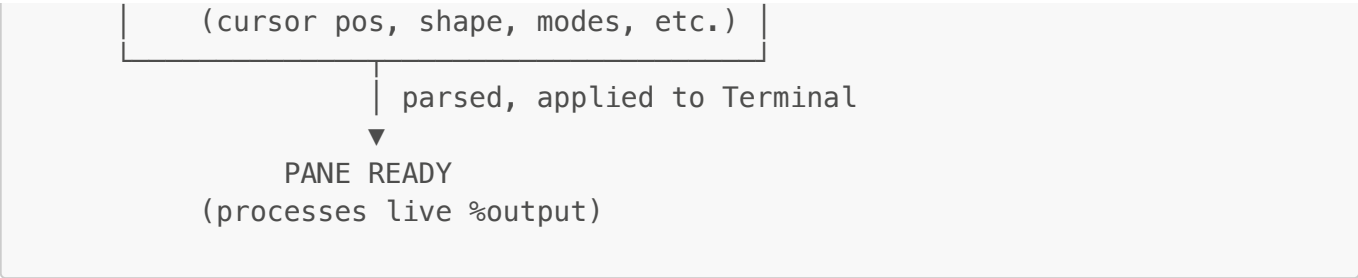




2.3 Per-Pane Initialization Sequence

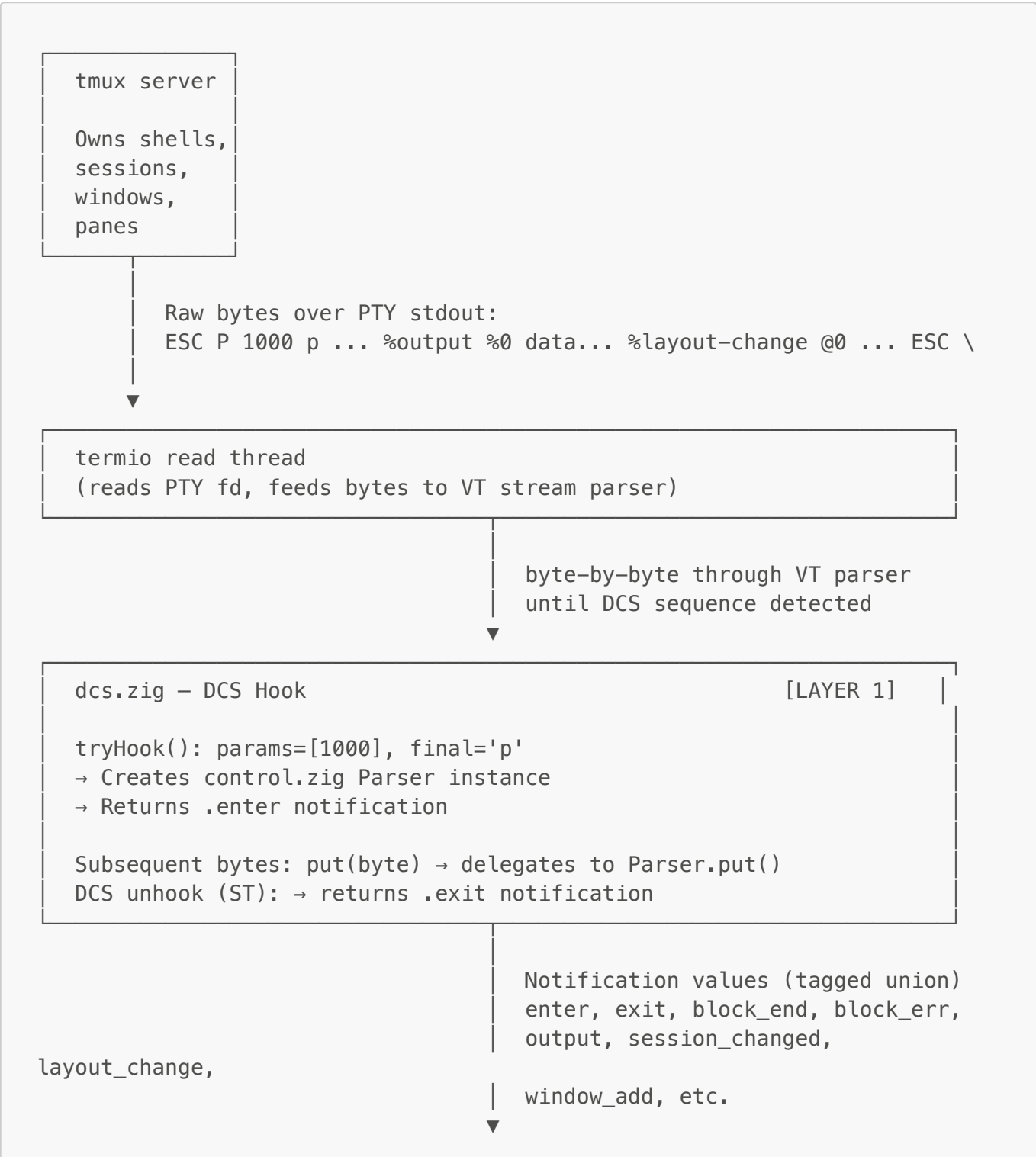
When a new pane is discovered (via `list-windows`), the Viewer queues 5 commands to fully initialize it:

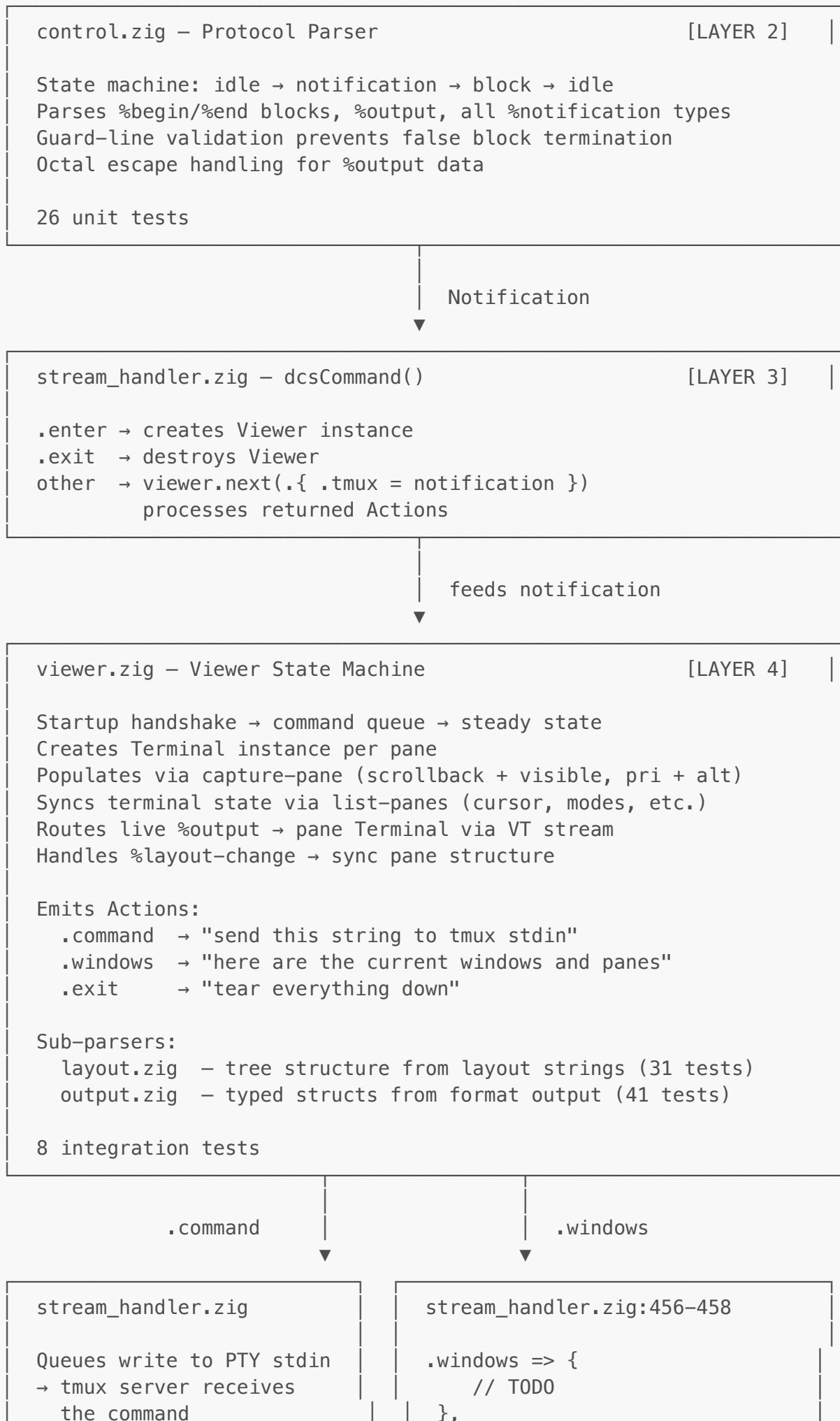


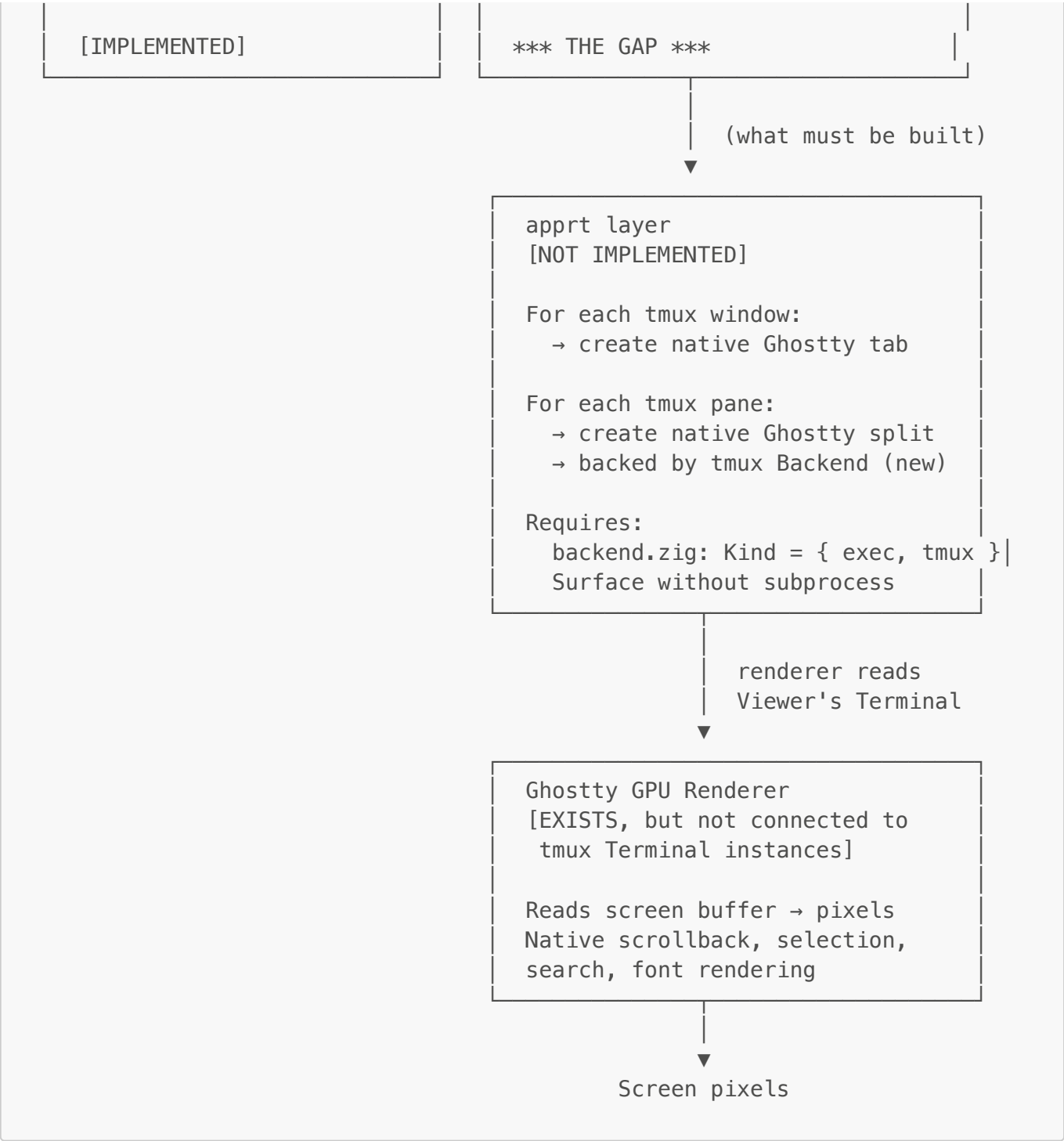


3. Data Flow: DCS Bytes to Screen Pixels

This shows the full pipeline from tmux server output through every layer of Ghostty to rendered pixels, highlighting the gap.

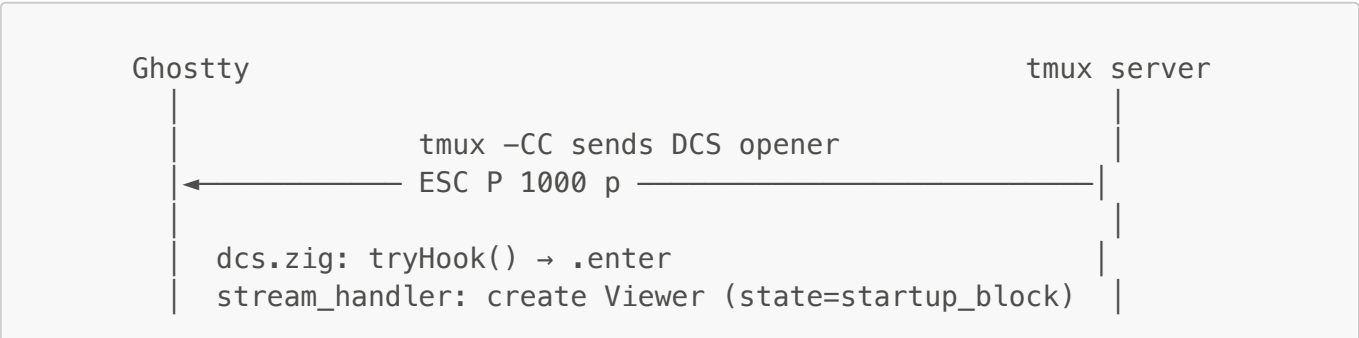


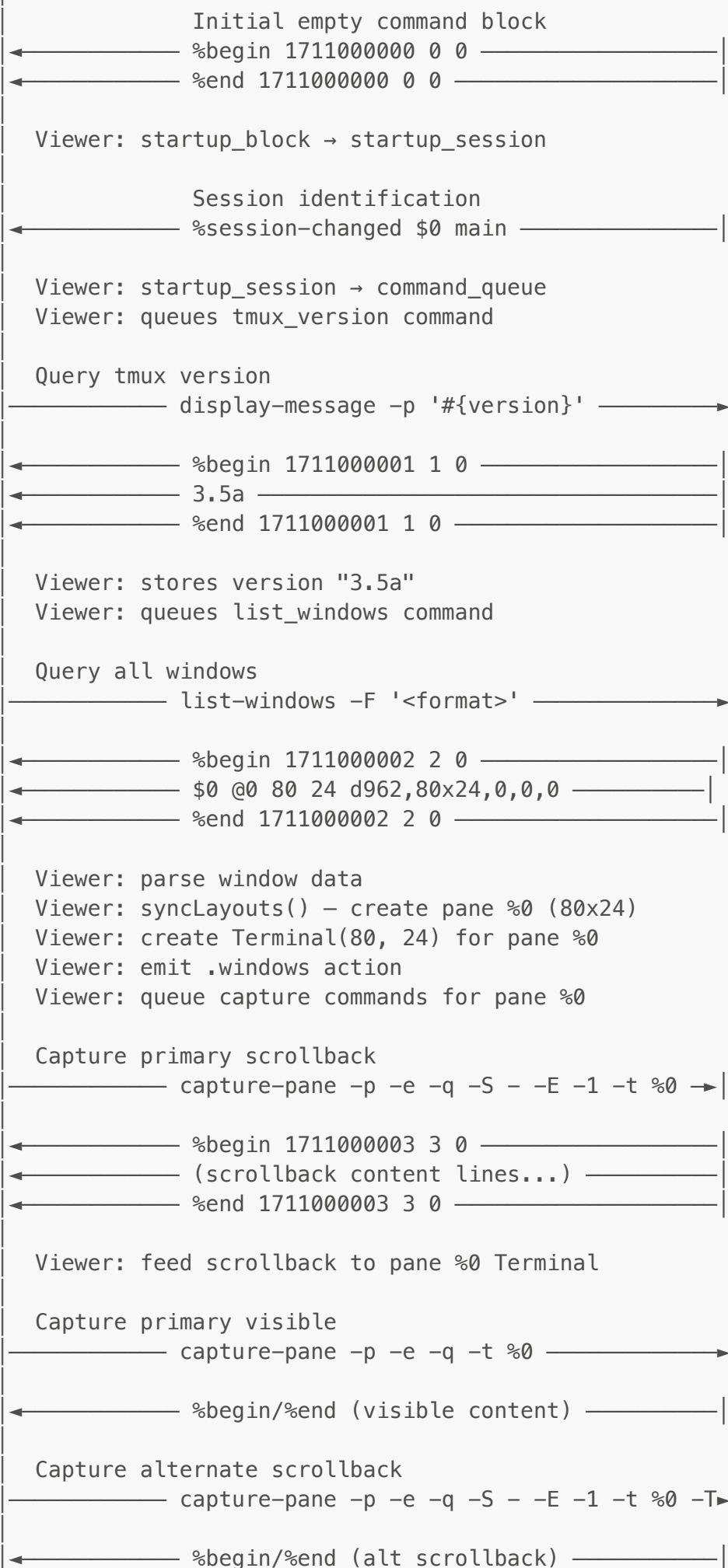


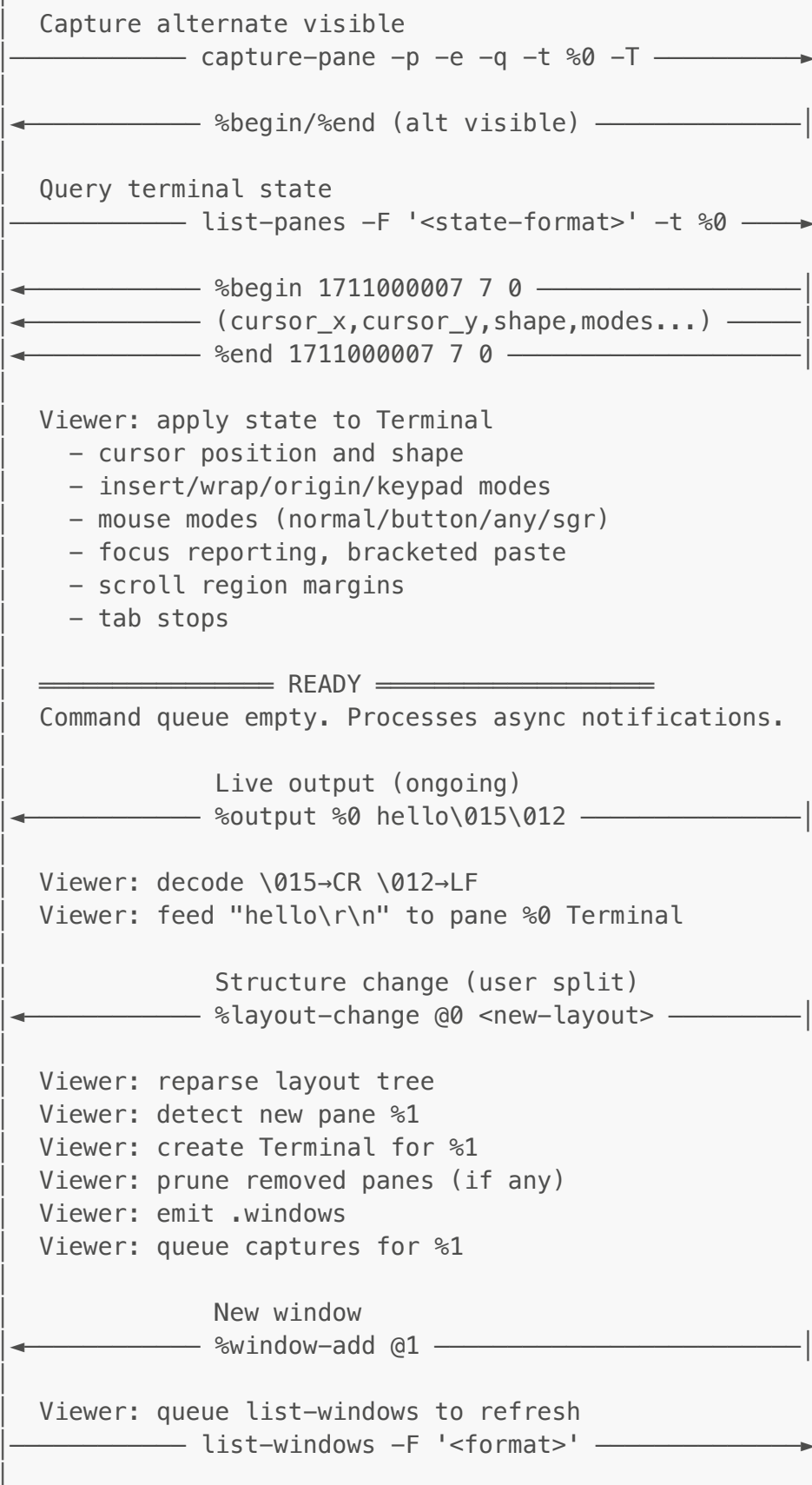


4. Command-Response Sequence Diagram

Temporal ordering of the complete startup handshake, the most fragile part of the protocol. Each arrow shows direction and content.







5. Layout Tree Examples

tmux describes pane arrangement as a layout string. The layout parser ([layout.zig](#)) turns these into a tree.

5.1 Single Pane

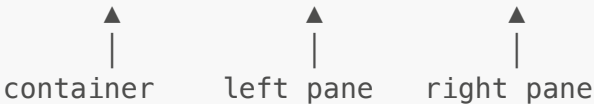
Layout string: "d962,80x24,0,0,0"
Checksum: d962 (CRC16 of "80x24,0,0,0")

Tree:

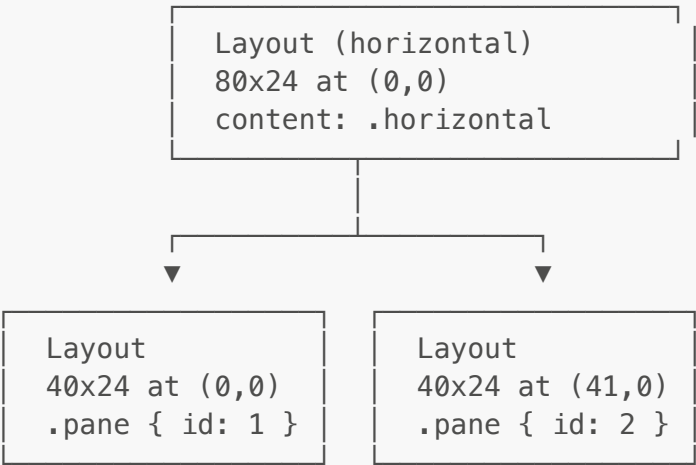


5.2 Horizontal Split (side by side)

Layout string: "a]b2,80x24,0,0,{40x24,0,0,1,40x24,41,0,2}"



Tree:

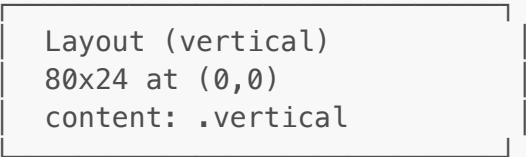


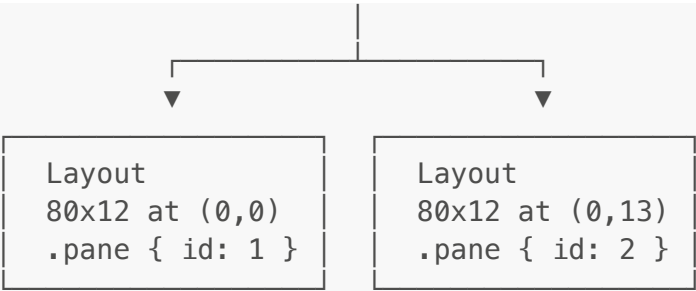
5.3 Vertical Split (stacked)

Layout string: "2f5e,80x24,0,0,[80x12,0,0,1,80x12,0,13,2]"



Tree:



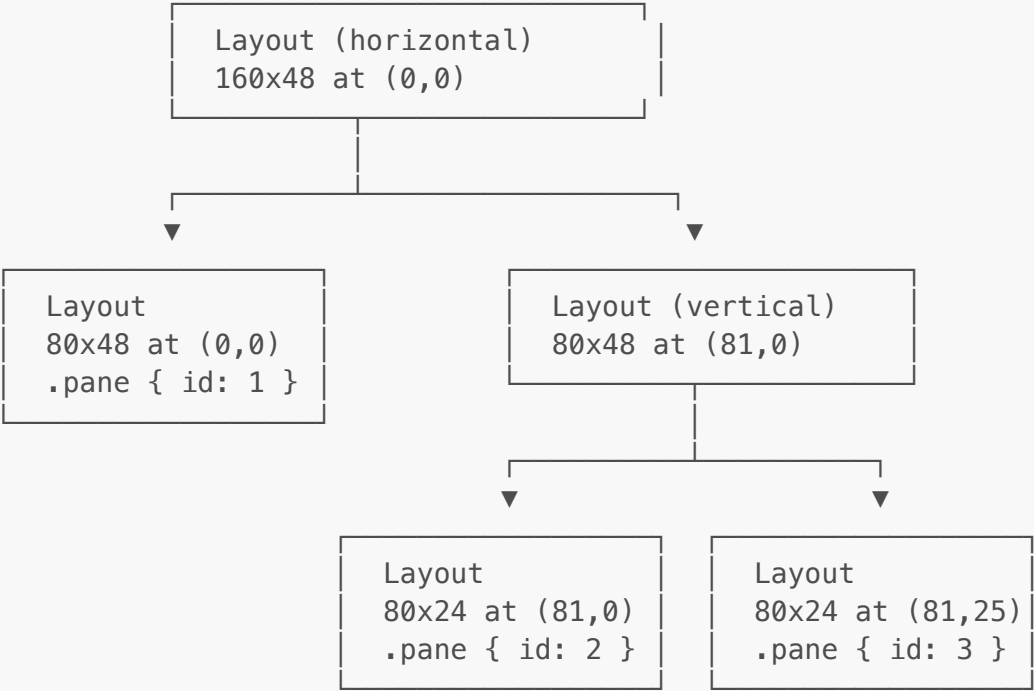


5.4 Complex Nested Layout

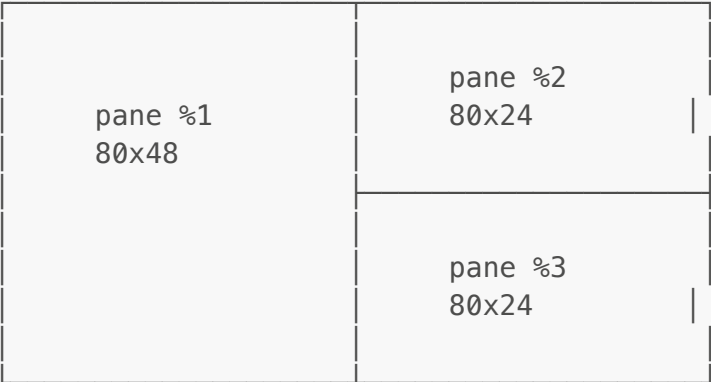
Layout string:
"8901,160x48,0,0{80x48,0,0,1,80x48,81,0[80x24,81,0,2,80x24,81,25,3]}"

Meaning: left pane full-height, right side split top/bottom

Tree:



Visual representation:



6. Protocol Trace Timeline

A sample timeline for debugging, showing the temporal ordering of events with states.

Time Action	Event Emitted	Viewer State
T+0.000	DCS hook: params=[1000] final=p	(not created yet)
—		
T+0.000	Notification: .enter create Viewer	—
T+0.001	Viewer created	startup_block
—		
T+0.012	Notification: .block_end("")	startup_block
—		
T+0.012	State transition	startup_session
—		
T+0.018	Notification: .session_changed(\$0,main)	startup_session
—		
T+0.018	State transition	command_queue
	.command("display-message...")	
T+0.025	Notification: .block_end("3.5a")	command_queue
	.command("list-windows...")	
T+0.032	Notification: .block_end("\$0 @0 80 ...")	command_queue
	.windows([{@0, 80x24, [%0]}])	
	↳ syncLayouts: create pane %0	
	.command("capture-pane...")	
T+0.040	Notification: .block_end(<scrollback>)	command_queue
	.command("capture-pane...")	
	↳ receivedPaneHistory: feed to Terminal	
T+0.048	Notification: .block_end(<visible>)	command_queue
	.command("capture-pane -T...")	
	↳ receivedPaneVisible: feed to Terminal	
T+0.055	Notification: .block_end(<alt scroll>)	command_queue
	.command("capture-pane -T...")	
	↳ receivedPaneHistory (alt screen)	
T+0.062	Notification: .block_end(<alt visible>)	command_queue
	.command("list-panes...")	
	↳ receivedPaneVisible (alt screen)	
T+0.070	Notification: .block_end(<state>)	command_queue
—		
	↳ receivedPaneState: apply cursor, modes, scroll region, tabs	
T+0.070	Command queue empty	command_queue
— (READY)		
T+1.500	Notification: .output(%0, "\$ ls\r\n")	command_queue
—		
	↳ receivedOutput: feed to pane %0 Term	

```

T+3.200      Notification: .layout_change(@0, ...)      command_queue
.window({{@0,...[%0,%1]}})
    ↳ layoutChanged: new pane %1 found
.command("capture-pane...")
T+3.210      Notification: .block_end(<pane1 hist>)      command_queue
.command("capture-pane...")
    ... (4 captures + state for pane %1)
T+5.000      Notification: .window_add(@1)              command_queue
.command("list-windows...")
    ↳ queues list_windows refresh
T+5.010      Notification: .block_end(<windows>)         command_queue
.window({{@0,...},{@1,...}})
    ↳ syncLayouts for both windows
.command("capture-pane...")

```

7. Notification Type Coverage Map

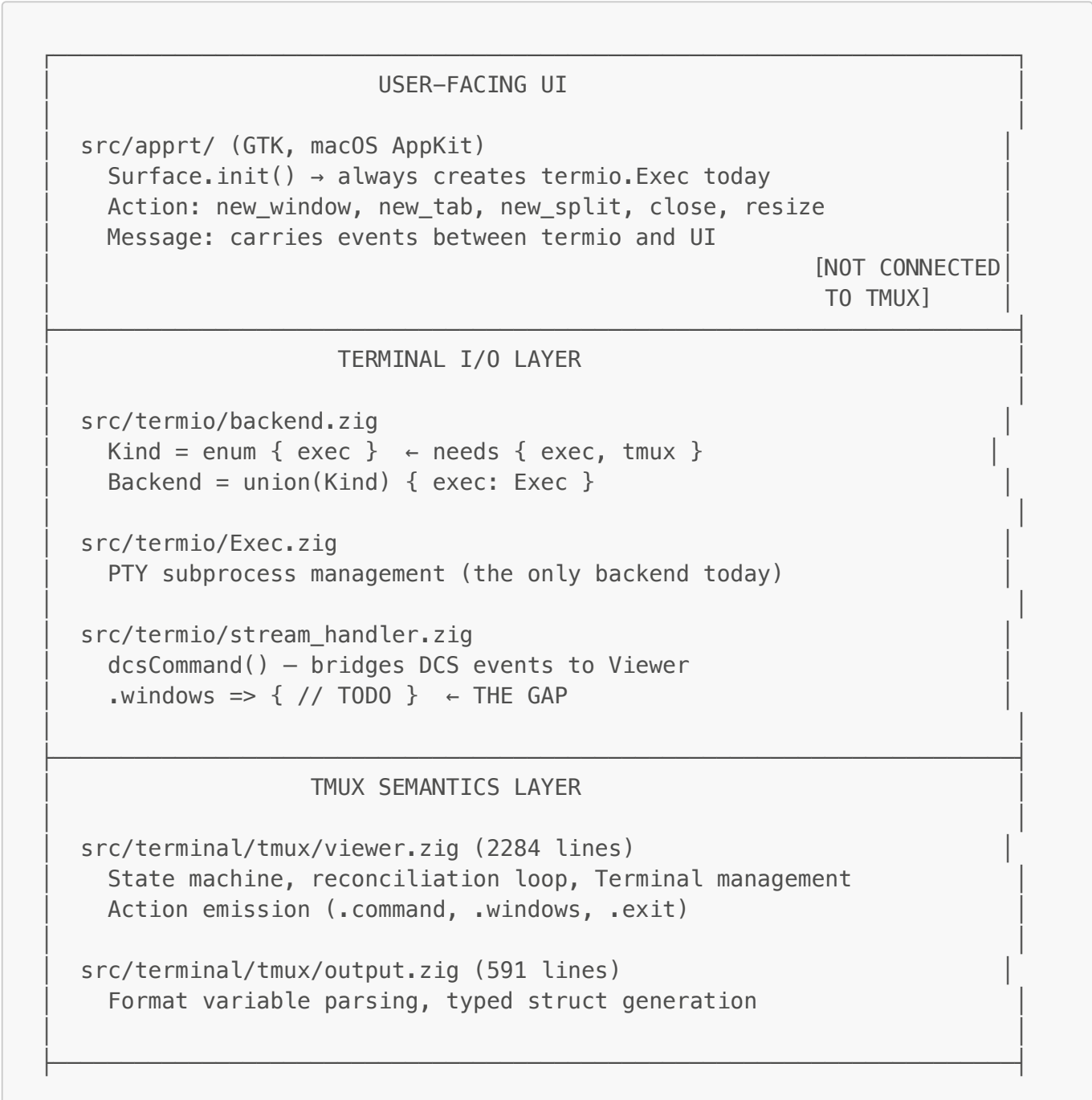
Which protocol notifications are handled at each layer.

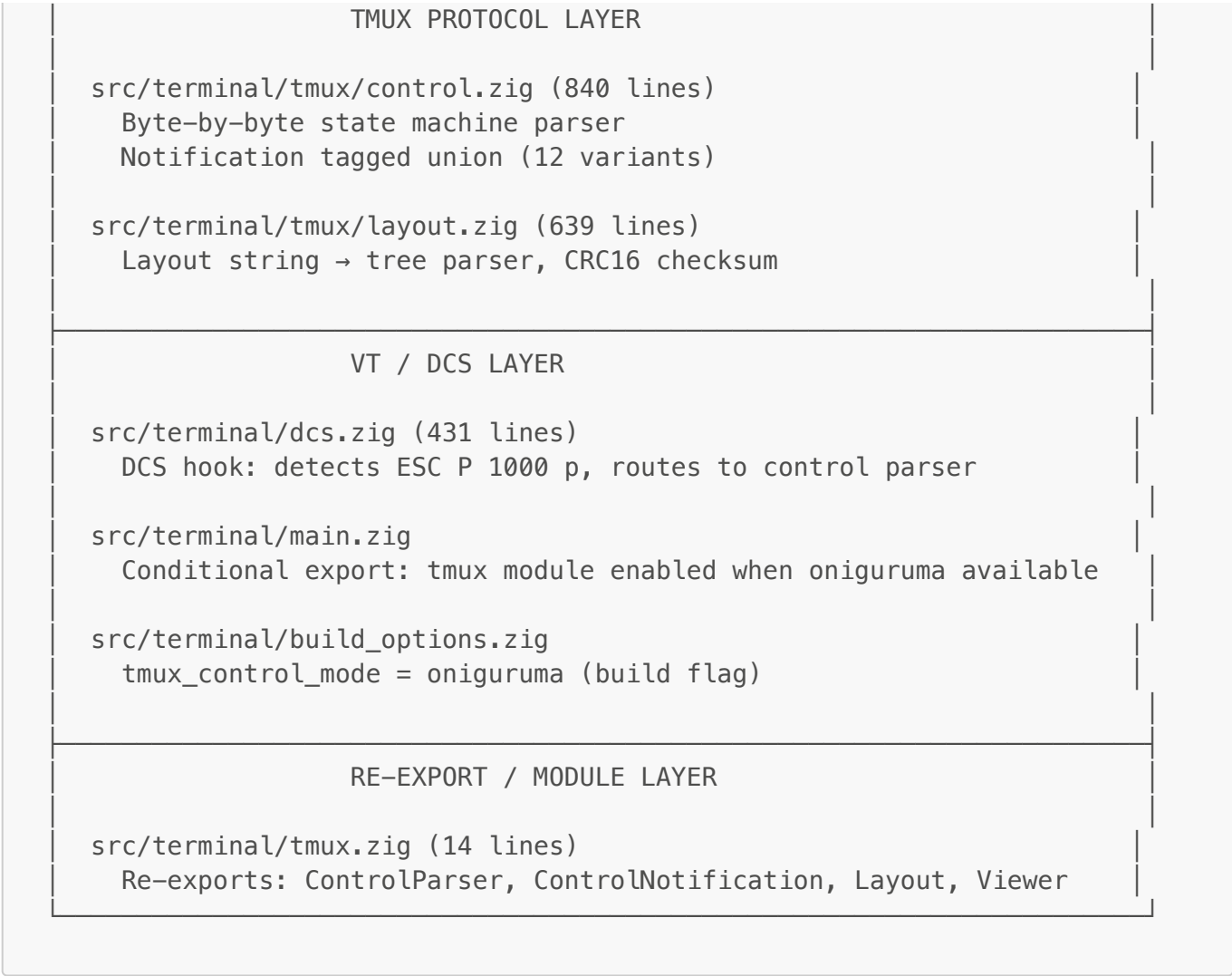
Protocol Message	Parser (control)	Viewer (viewer)	stream_handler (glue)	GUI (apprt)
%begin/%end/%error	YES	YES	YES	—
%output	YES	YES	(via viewer)	—
%extended-output	NO	—	—	—
%session-changed	YES	YES	(via viewer)	—
%session-renamed	NO	—	—	—
%sessions-changed	YES	IGNORED	—	—
%session-window-changed	NO	—	—	—
%window-add	YES	YES	(via viewer)	—
%window-close	NO	—	—	—
%window-renamed	YES	IGNORED	—	—
%window-pane-changed	YES	IGNORED	—	—
%layout-change	YES	YES	(via viewer)	—
%client-detached	YES	IGNORED	—	—
%client-session-changed	YES	IGNORED	—	—
%paste-buffer-changed	NO	—	—	—
%paste-buffer-deleted	NO	—	—	—
%pane-mode-changed	NO	—	—	—
%pause	NO	—	—	—
%continue	NO	—	—	—
%subscription-changed	NO	—	—	—
%unlinked-window-*	NO	—	—	—
%exit	YES	YES	YES	—
Commands Sent	Parser	Viewer	stream_handler	GUI
display-message	—	YES	YES (writes)	—
list-windows	—	YES	YES (writes)	—
list-panes	—	YES	YES (writes)	—

capture-pane	—	YES	YES (writes)	—
send-keys	—	NO	—	NOT IMPL
refresh-client -C	—	NO	—	NOT IMPL
new-window	—	NO	—	NOT IMPL
split-window	—	NO	—	NOT IMPL
kill-window	—	NO	—	NOT IMPL
kill-pane	—	NO	—	NOT IMPL
resize-pane	—	NO	—	NOT IMPL
select-window	—	NO	—	NOT IMPL
select-pane	—	NO	—	NOT IMPL
refresh-client -f	—	NO	—	NOT IMPL

8. Architecture Layer Map

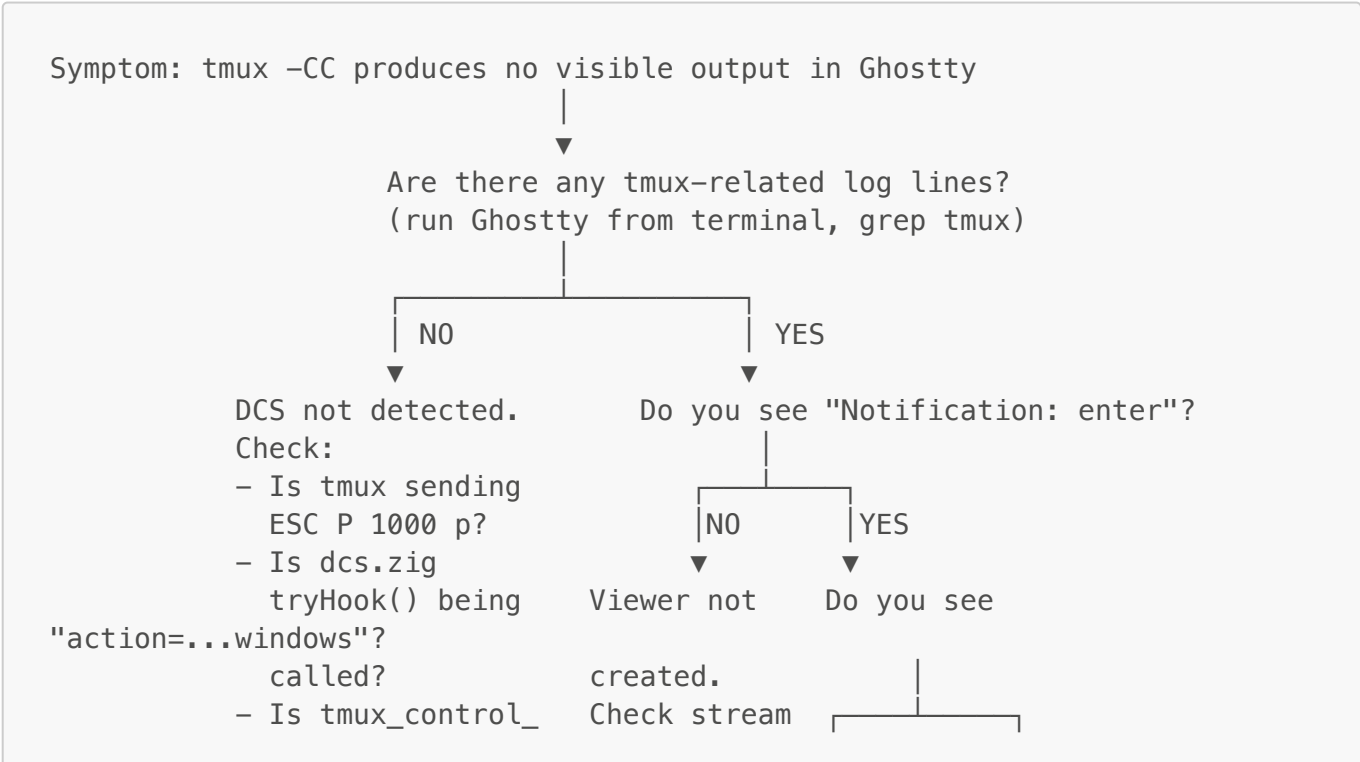
How the source files organize into layers.

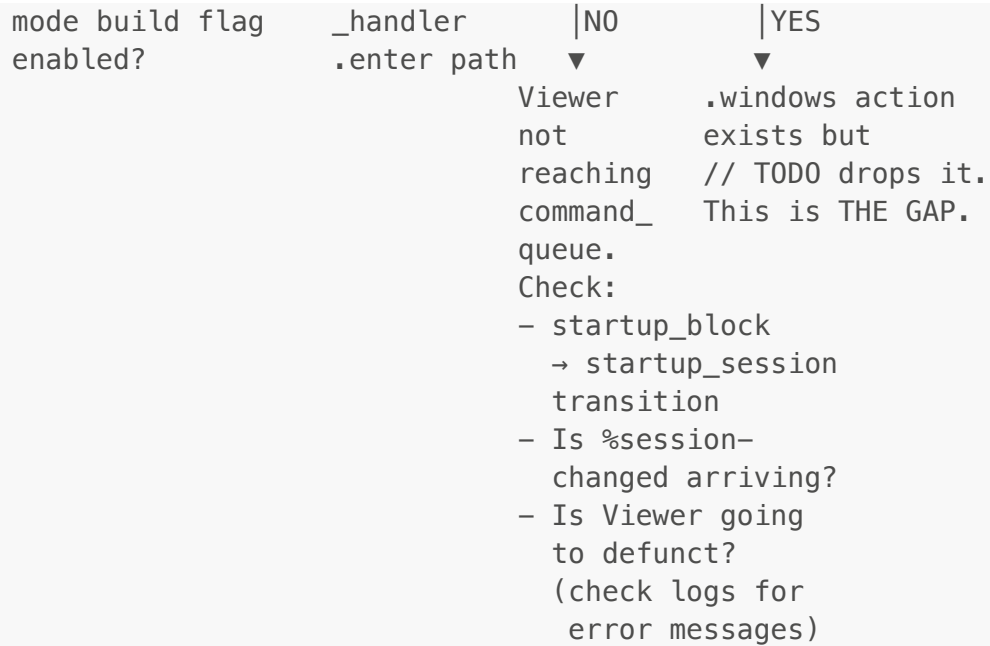




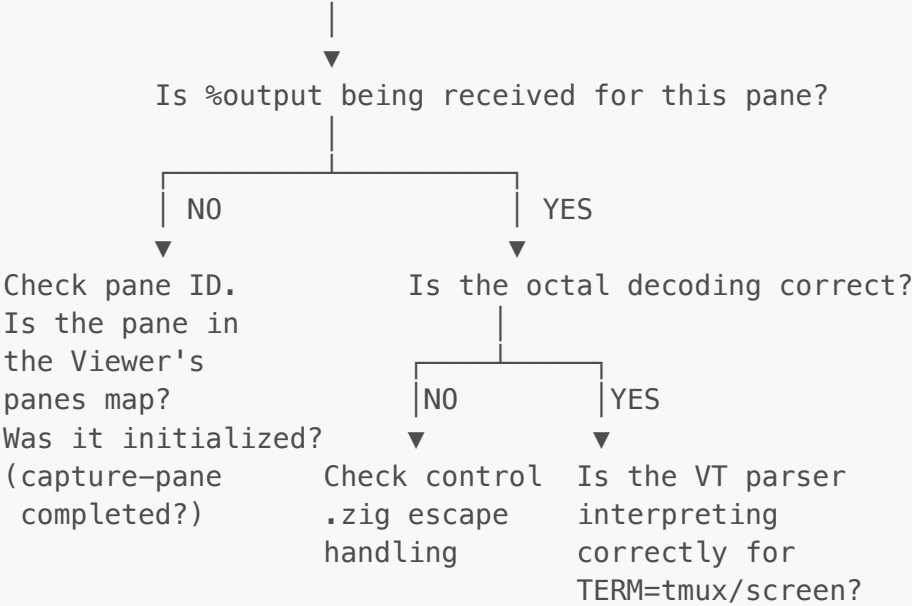
9. Debugging Decision Tree

When something goes wrong, use this to identify which layer to investigate.





Symptom: Wrong pane content



Symptom: Viewer enters defunct state

